BusNStay Flutter App - Complete Step-by-Step Implementation Guide

📋 Table of Contents

1. Project Setup
2. File Structure
3. Implementation
4. Testing
5. Deployment

STEP 1: Project Setup

Option A: Using CLI (Recommended for Windows)

```
# Open PowerShell and run:
cd C:\Users\zwexm\Projects

# Create Flutter project
flutter create --org com.busnstay --project-name busnstay_app busnstay_flutter

cd busnstay_flutter

# Get dependencies
flutter pub get

# Verify installation
flutter doctor
```

Option B: Android Studio

- File → New Flutter Project
- Select Flutter SDK path
- Create project with name: `busnstay_flutter`
- Package name: `com.busnstay.app`

STEP 2: Update pubspec.yaml

Replace the entire `pubspec.yaml` with:

```
name: busnstay_flutter
description: "BusNStay - African Transportation & Delivery Platform"
publish_to: 'none'
```

 Copyright (c) Zwesta Empire All rights reserved.

```yaml
version: 1.0.0+1

environment:
  sdk: '>=3.0.0 <4.0.0'

dependencies:
  flutter:
    sdk: flutter
  cupertino_icons: ^1.0.2

  #🎯 State Management (Riverpod - recommended over Provider)
  flutter_riverpod: ^2.4.0
  riverpod_generator: ^2.3.0

  # 🌐 API & Backend
  supabase_flutter: ^1.10.0
  dio: ^5.3.0

  # 🎨 UI Components
  flutter_screenutil: ^5.9.0
  google_fonts: ^6.1.0
  cached_network_image: ^3.3.0

  # 📍 Maps & Location
  google_maps_flutter: ^2.5.0
  geolocator: ^10.1.0

  # 💳 Payment
  flutterwave_standard: ^1.0.0

  # 💾 Local Storage
  shared_preferences: ^2.2.0
  get_storage: ^2.1.0

  # 🔔 Notifications
  firebase_core: ^2.24.0
  firebase_messaging: ^14.6.0
  firebase_analytics: ^10.7.0

  # 📅 Date & Time
  intl: ^0.19.0
  table_calendar: ^3.0.0

  # 🛠️ Utilities
  uuid: ^4.0.0
  get: ^4.6.0
  connectivity_plus: ^5.0.0
  url_launcher: ^6.2.0
  share_plus: ^7.1.0
  image_picker: ^1.0.0

dev_dependencies:
```

 Copyright (c) Zwesta Empire All rights reserved.

```yaml
  flutter_test:
    sdk: flutter
  flutter_lints: ^3.0.0
  build_runner: ^2.4.0
  riverpod_generator: ^2.3.0

flutter:
  uses-material-design: true
  assets:
    - assets/images/
    - assets/icons/
    - assets/animations/

  fonts:
    - family: Inter
      fonts:
        - asset: assets/fonts/Inter-Regular.ttf
        - asset: assets/fonts/Inter-Bold.ttf
          weight: 700
        - asset: assets/fonts/Inter-SemiBold.ttf
          weight: 600
    - family: SpaceGrotesk
      fonts:
        - asset: assets/fonts/SpaceGrotesk-Regular.ttf
        - asset: assets/fonts/SpaceGrotesk-Bold.ttf
          weight: 700
```

STEP 3: Create Directory Structure

```bash
# In PowerShell, from project root:
mkdir lib/config
mkdir lib/models
mkdir lib/services
mkdir lib/providers
mkdir lib/screens/auth
mkdir lib/screens/home
mkdir lib/screens/restaurant
mkdir lib/screens/hotel
mkdir lib/screens/delivery
mkdir lib/screens/account
mkdir lib/screens/admin
mkdir lib/widgets
mkdir lib/utils
mkdir lib/l10n
mkdir assets/{images,icons,animations,fonts}
mkdir test
```

 Copyright (c) Zwesta Empire All rights reserved.

STEP 4: Core Configuration Files

lib/config/constants.dart

```
// Copy from FLUTTER_APP_COMPLETE_GUIDE.md
```

lib/config/theme.dart

```
// Copy from FLUTTER_APP_COMPLETE_GUIDE.md
```

lib/main.dart

```dart
import 'package:flutter/material.dart';
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:flutter_screenutil/flutter_screenutil.dart';
import 'package:supabase_flutter/supabase_flutter.dart';
import 'config/theme.dart';
import 'config/constants.dart';
import 'screens/auth/login_screen.dart';
import 'screens/splash_screen.dart';

void main() async {
  WidgetsFlutterBinding.ensureInitialized();

  await Supabase.initialize(
    url: AppConstants.supabaseUrl,
    anonKey: AppConstants.supabaseAnonKey,
  );

  runApp(
    const ProviderScope(
      child: BusNStayApp(),
    ),
  );
}

class BusNStayApp extends StatelessWidget {
  const BusNStayApp({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return ScreenUtilInit(
      designSize: const Size(375, 812),
      minTextAdapt: true,
      splitScreenMode: true,
      builder: (context, child) {
```

 Copyright (c) Zwesta Empire All rights reserved.

```
        return MaterialApp(
          title: AppConstants.appName,
          theme: BusNStayTheme.getLightTheme(),
          darkTheme: BusNStayTheme.getDarkTheme(),
          themeMode: ThemeMode.system,
          home: child,
        );
      },
      child: const LoginScreen(),
    );
  }
}
```

STEP 5: Create Models

lib/models/user.dart

```
import 'package:freezed_annotation/freezed_annotation.dart';

part 'user.freezed.dart';
part 'user.g.dart';

@freezed
class User with _$User {
  const factory User({
    required String id,
    required String email,
    required String fullName,
    required String phoneNumber,
    String? profileImageUrl,
    @Default('active') String accountStatus,
    @Default(false) bool isEmailVerified,
    @Default(false) bool isPhoneVerified,
    @Default(false) bool twoFactorEnabled,
    required DateTime createdAt,
    required DateTime updatedAt,
  }) = _User;

  factory User.fromJson(Map<String, dynamic> json) => _$UserFromJson(json);
}

@freezed
class UserRole with _$UserRole {
  const factory UserRole({
    required String id,
    required String userId,
    required String role,
    @Default(true) bool isActive,
    String? verificationStatus,
```

 Copyright (c) Zwesta Empire All rights reserved.

```
    DateTime? verifiedAt,
    Map<String, dynamic>? metadata,
  }) = _UserRole;

  factory UserRole.fromJson(Map<String, dynamic> json) =>
_$UserRoleFromJson(json);
}
```

lib/models/journey.dart

```
import 'package:freezed_annotation/freezed_annotation.dart';
import 'package:geolocator/geolocator.dart';

part 'journey.freezed.dart';
part 'journey.g.dart';

@freezed
class Journey with _$Journey {
  const factory Journey({
    required String id,
    required String routeId,
    required String busId,
    required DateTime departureTime,
    required DateTime arrivalTime,
    required String departurePoint,
    required String destinationPoint,
    required double fare,
    @Default('scheduled') String status,
    required int totalSeats,
    required int availableSeats,
    required DateTime createdAt,
  }) = _Journey;

  factory Journey.fromJson(Map<String, dynamic> json) =>
_$JourneyFromJson(json);
}

@freezed
class Bus with _$Bus {
  const factory Bus({
    required String id,
    required String operatorId,
    required String licensePlate,
    required String model,
    required int seatsTotal,
    @Default([]) List<String> amenities,
    required String status,
    required DateTime createdAt,
  }) = _Bus;
```

 Copyright (c) Zwesta Empire All rights reserved.

```
  factory Bus.fromJson(Map<String, dynamic> json) => _$BusFromJson(json);
}

@freezed
class Booking with _$Booking {
  const factory Booking({
    required String id,
    required String journeyId,
    required String passengerId,
    required List<int> seatNumbers,
    required double totalPrice,
    @Default('confirmed') String status,
    required DateTime bookingTime,
    DateTime? cancelledAt,
  }) = _Booking;

  factory Booking.fromJson(Map<String, dynamic> json) =>
_$BookingFromJson(json);
}
```

lib/models/restaurant.dart

```
import 'package:freezed_annotation/freezed_annotation.dart';

part 'restaurant.freezed.dart';
part 'restaurant.g.dart';

@freezed
class Restaurant with _$Restaurant {
  const factory Restaurant({
    required String id,
    required String name,
    required String description,
    String? imageUrl,
    required double rating,
    required int reviewCount,
    required String cuisine,
    required String openingHours,
    required double deliveryFee,
    required int deliveryTime,
    @Default('open') String status,
  }) = _Restaurant;

  factory Restaurant.fromJson(Map<String, dynamic> json) =>
_$RestaurantFromJson(json);
}

@freezed
class MenuItem with _$MenuItem {
  const factory MenuItem({
```

 Copyright (c) Zwesta Empire All rights reserved.

```
    required String id,
    required String restaurantId,
    required String name,
    required String description,
    String? imageUrl,
    required double price,
    @Default([]) List<String> category,
    @Default(true) bool available,
  }) = _MenuItem;

  factory MenuItem.fromJson(Map<String, dynamic> json) =>
_$MenuItemFromJson(json);
}

@freezed
class Order with _$Order {
  const factory Order({
    required String id,
    required String restaurantId,
    required String customerId,
    required List<OrderItem> items,
    required double subtotal,
    required double deliveryFee,
    required double tax,
    required double total,
    @Default('pending') String status,
    String? deliveryAddress,
    String? notes,
    required DateTime createdAt,
  }) = _Order;

  factory Order.fromJson(Map<String, dynamic> json) => _$OrderFromJson(json);
}

@freezed
class OrderItem with _$OrderItem {
  const factory OrderItem({
    required String id,
    required String menuItemId,
    required String name,
    required double price,
    required int quantity,
  }) = _OrderItem;

  factory OrderItem.fromJson(Map<String, dynamic> json) =>
_$OrderItemFromJson(json);
}
```

STEP 6: Create Services

Copyright (c) Zwesta Empire All rights reserved.

lib/services/auth_service.dart

```
import 'package:supabase_flutter/supabase_flutter.dart';
import 'package:shared_preferences/shared_preferences.dart';

class AuthService {
  final SupabaseClient _supabase;
  late SharedPreferences _prefs;

  AuthService(this._supabase);

  Future<void> initialize() async {
    _prefs = await SharedPreferences.getInstance();
  }

  // Register new user
  Future<Map<String, dynamic>> register({
    required String email,
    required String password,
    required String fullName,
    String? phoneNumber,
    required bool termsAccepted,
  }) async {
    try {
      final response = await _supabase.auth.signUp(
        email: email,
        password: password,
      );

      if (response.user == null) {
        return {'success': false, 'error': 'User creation failed'};
      }

      // Create user profile
      await _supabase.from('users').insert({
        'id': response.user!.id,
        'email': email,
        'username': email.split('@')[0],
        'full_name': fullName,
        'phone_number': phoneNumber,
        'terms_accepted': termsAccepted,
      });

      // Assign passenger role
      await _supabase.from('user_roles').insert({
        'user_id': response.user!.id,
        'role': 'passenger',
      });

      // Create passenger profile
      await _supabase.from('passenger_profiles').insert({
```

 Copyright (c) Zwesta Empire All rights reserved.

```
        'user_id': response.user!.id,
      });

      return {'success': true, 'userId': response.user!.id};
    } catch (e) {
      return {'success': false, 'error': e.toString()};
    }
  }

  // Login
  Future<Map<String, dynamic>> login({
    required String email,
    required String password,
  }) async {
    try {
      final response = await _supabase.auth.signInWithPassword(
        email: email,
        password: password,
      );

      if (response.user == null) {
        return {'success': false, 'error': 'Login failed'};
      }

      await _prefs.setString('userId', response.user!.id);

      return {'success': true, 'userId': response.user!.id};
    } catch (e) {
      return {'success': false, 'error': e.toString()};
    }
  }

  // Logout
  Future<void> logout() async {
    await _supabase.auth.signOut();
    await _prefs.remove('userId');
  }

  bool get isAuthenticated => _supabase.auth.currentUser != null;
  String? get userId => _supabase.auth.currentUser?.id;
}
```

lib/services/journey_service.dart

```
import 'package:supabase_flutter/supabase_flutter.dart';
import '../models/journey.dart';

class JourneyService {
  final SupabaseClient _supabase;
```

 Copyright (c) Zwesta Empire All rights reserved.

```
  JourneyService(this._supabase);

  Future<List<Journey>> searchJourneys({
    required String from,
    required String to,
    required DateTime date,
  }) async {
    try {
      final response = await _supabase
          .from('journeys')
          .select()
          .eq('departure_point', from)
          .eq('destination_point', to)
          .gte('departure_time', date.toIso8601String())
          .lte('departure_time', date.add(const Duration(days:
1)).toIso8601String());

      return (response as List)
          .map((e) => Journey.fromJson(e))
          .toList();
    } catch (e) {
      rethrow;
    }
  }

  Future<Journey?> getJourneyDetails(String journeyId) async {
    try {
      final response = await _supabase
          .from('journeys')
          .select()
          .eq('id', journeyId)
          .single();

      return Journey.fromJson(response);
    } catch (e) {
      return null;
    }
  }

  Future<bool> bookJourney({
    required String journeyId,
    required List<int> seatNumbers,
    required double totalPrice,
  }) async {
    try {
      await _supabase.from('journey_passengers').insert({
        'journey_id': journeyId,
        'passenger_id': _supabase.auth.currentUser!.id,
        'seat_numbers': seatNumbers,
        'total_price': totalPrice,
        'status': 'confirmed',
      });
```

 Copyright (c) Zwesta Empire All rights reserved.

```
      return true;
    } catch (e) {
      return false;
    }
  }
}
```

STEP 7: Create Providers (State Management)

lib/providers/auth_provider.dart

```
import 'package:flutter_riverpod/flutter_riverpod.dart';
import 'package:supabase_flutter/supabase_flutter.dart';
import '../services/auth_service.dart';

final supabaseProvider = Provider((ref) => Supabase.instance.client);

final authServiceProvider = Provider((ref) {
  final supabase = ref.watch(supabaseProvider);
  return AuthService(supabase);
});

final authStateProvider = StreamProvider((ref) {
  return Supabase.instance.client.auth.onAuthStateChange;
});

final loginProvider = FutureProvider.family<
    Map<String, dynamic>,
    ({String email, String password})>((ref, params) async {
  final authService = ref.watch(authServiceProvider);
  return authService.login(
    email: params.email,
    password: params.password,
  );
});

class AuthNotifier extends StateNotifier<AsyncValue<User?>> {
  final AuthService _authService;

  AuthNotifier(this._authService) : super(const AsyncValue.loading()) {
    _init();
  }

  Future<void> _init() async {
    // Load current user
    final user = Supabase.instance.client.auth.currentUser;
    if (user != null) {
      state = AsyncValue.data(User(
        id: user.id,
```

 Copyright (c) Zwesta Empire All rights reserved.

```
        email: user.email ?? '',
        fullName: '',
        phoneNumber: '',
        createdAt: user.createdAt ?? DateTime.now(),
        updatedAt: DateTime.now(),
      ));
    } else {
      state = const AsyncValue.data(null);
    }
  }
}
```

STEP 8: Create Screens

lib/screens/auth/login_screen.dart

```
// Copy from FLUTTER_APP_COMPLETE_GUIDE.md - lib/screens/auth/login_screen.dart
```

lib/screens/splash_screen.dart

```
import 'package:flutter/material.dart';
import 'package:flutter_screenutil/flutter_screenutil.dart';
import '../config/theme.dart';

class SplashScreen extends StatelessWidget {
  const SplashScreen({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      backgroundColor: BusNStayTheme.primary,
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            Text(
              'BusNStay',
              style: Theme.of(context).textTheme.displayLarge?.copyWith(
                    color: Colors.white,
              ),
            ),
            SizedBox(height: 32.h),
            SizedBox(
              width: 50.w,
              height: 50.w,
              child: const CircularProgressIndicator(
                color: Colors.white,
```

 Copyright (c) Zwesta Empire All rights reserved.

```
              ),
            ),
          ],
        ),
      ),
    );
  }
}
```

STEP 9: Build & Test

```
# Get dependencies
flutter pub get

# Run on emulator
flutter run

# Build release APK
flutter build apk --release

# Build AAB (Google Play)
flutter build appbundle --release

# View file size
flutter build apk --release --analyze-size
```

STEP 10: Firebase Setup

Generate Firebase Configuration

```
# Install Firebase CLI if not already installed
npm install -g firebase-tools

# Login to Firebase
firebase login

# Configure Flutter for Firebase
flutterfire configure

# Select project: BusNStay
```

This will generate:

- `google-services.json` (Android)

 Copyright (c) Zwesta Empire All rights reserved.

- `GoogleService-Info.plist` (iOS)

STEP 11: Configure Android for Payments

android/app/build.gradle

```
android {
    compileSdkVersion 34

    defaultConfig {
        applicationId "com.busnstay.app"
        minSdkVersion 21
        targetSdkVersion 34
        versionCode 1
        versionName "1.0.0"
    }
}

dependencies {
    // Flutterwave
    implementation 'com.flutterwave.raveandroid:rave-android-sdk:1.0.80'

    // Location
    implementation 'com.google.android.gms:play-services-location:21.0.1'

    // Maps
    implementation 'com.google.android.gms:play-services-maps:18.1.0'
}
```

AndroidManifest.xml

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <!-- Permissions -->
    <uses-permission android:name="android.permission.INTERNET" />
    <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
    <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION"
/>
    <uses-permission android:name="android.permission.CAMERA" />
    <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE" />

    <application>
        <!-- Google Maps API Key -->
        <meta-data
            android:name="com.google.android.geo.API_KEY"
            android:value="YOUR_GOOGLE_MAPS_API_KEY" />
    </application>
</manifest>
```

 Copyright (c) Zwesta Empire All rights reserved.

STEP 12: Deploy to Play Store

```
# Create keystore
keytool -genkey -v -keystore ~/release-key.jks -keyalg RSA -keysize 2048 -
validity 10000 -alias busnstay

# Build signed APK
flutter build apk --release

# Build AAB for Play Store (recommended)
flutter build appbundle --release

# APK location: build/app/outputs/apk/release/app-release.apk
# AAB location: build/app/outputs/bundle/release/app-release.aab
```

Then upload to Google Play Console.

Final Checklist

- ☐ Create Flutter project
- ☐ Update pubspec.yaml
- ☐ Create all configuration files
- ☐ Create all model files
- ☐ Create all service files
- ☐ Create all provider files
- ☐ Create all screen files
- ☐ Update AndroidManifest.xml
- ☐ Setup Firebase
- ☐ Test on emulator
- ☐ Build release APK
- ☐ Deploy to Play Store
- ☐ Monitor analytics

Troubleshooting

Issue: "No Flutter SDK found"

```
flutter doctor
flutter doctor --android-licenses
```

Issue: "Dart SDK not installed"

Copyright (c) Zwesta Empire All rights reserved.

```
flutter --version
flutter upgrade
```

Issue: "Gradle build failed"

```
cd android
./gradlew clean
cd ..
flutter clean
flutter pub get
flutter run
```

Issue: "API key not valid"

- Check Supabase configuration in `constants.dart`
- Verify Google Maps API key in AndroidManifest.xml
- Check Flutterwave configuration

Resources

- Flutter Docs: https://docs.flutter.dev
- Supabase Docs: https://supabase.com/docs
- Riverpod Docs: https://riverpod.dev
- Flutterwave Docs: https://developer.flutterwave.com

Copyright (c) Zwesta Empire All rights reserved.